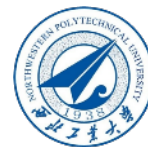


numpy



- ◆ 掌握ndarray多维数组的创建
- ◆ 掌握数组元素的索引、切片和选择
- ◆ 掌握数组的运算
- ◆ 掌握读写数据文件

➤ NumPy 的部分功能如下：

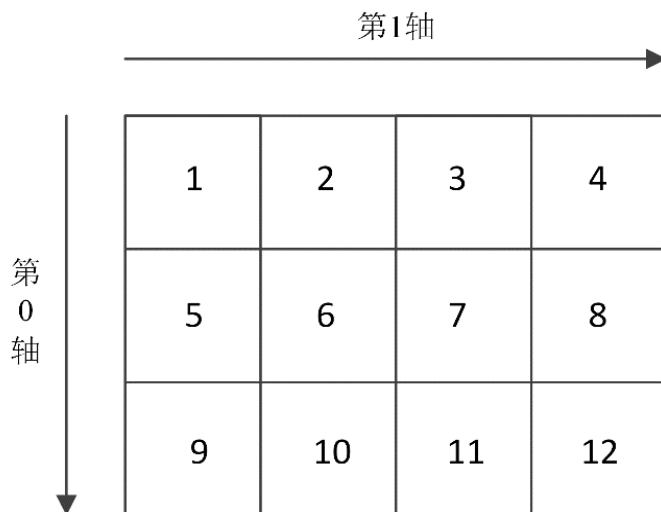
- (1) ndarray：一个具有**矢量算术运算**且**节省空间的多维数组**。
- (2) 用于对**整组数据**进行快速运算的标准数学函数(**无须编写循环**)。
- (3) 用于**读/写磁盘数据**的工具以及用于**操作内存映射文件**的工具。
- (4) 线性代数、随机数生成以及傅里叶变换功能。
- (5) 用于集成 C、C++、Fortran 等语言的代码编写工具。

NumPy提供了两种基本的对象：

- **ndarray**: 英文全称为n-dimensional array object，称为多维数组，后统一称之为**数组**。
- **ufunc**: 英文全称为universal function object，它是一种能够对**数组**进行**处理**的**特殊函数**。
- NumPy库的核心对象是n维数组对象ndarray，Python中所有的ufunc函数都是围绕ndarray对象进行的。ndarray数组能够对**整块数据**进行**数学运算**，通常来说，ndarray是存储单一数据的容器，即其中的所有元素都需要是相同的类型，和list不同，它能**直接保存数据**，而list保存的是对象的引用。

● NumPy多维数组

- NumPy常用的导入格式：`import numpy as np`。
- NumPy库的基础是**N维数组对象**（即ndarray对象），该对象由两部分组成，一部分是实际的数据（同种类型），另一部分是描述这些数据的元数据。
- ndarray数组中的维度（**dimensions**）又称为轴（**axis**）。
- 数组的第**0**轴对应着第一维，第**1**轴对应着第二维，维度为**(3,4)**的形状，元素总个数为**12**。



3×4数组的两个轴



➤ 创建ndarray对象:

通常来说，`ndarray`是一个通用的同构数据容器，即其中的所有元素都需要相同的类型。利用`array`函数可创建`ndarray`数组。

1. 利用array函数创建数组对象

- array函数的格式: `np.array(object, dtype, ndmin)`

array函数的主要参数及说明表

参数名称	说明
object	接收array, 表示想要创建的数组
dtype	接收data-type, 表示数组所需的数据类型, 未给定则选择保存对象所需的最小类型, 默认为None
ndmin	接收int, 制定生成数组应该具有的最小维数, 默认为None

1. 利用array函数创建数组对象

- array函数的格式: `np.array(object, dtype, ndmin)`

```
import numpy as np
data1 = [1, 3, 5, 7] #列表
w1 = np.array(data1)
print('w1:', w1)
data2 = (2, 4, 6, 8) #元组
w2 = np.array(data2)
print('w2:', w2)
data3 = [[1, 2, 3, 4], [5, 6, 7, 8]] #多维数组
w3 = np.array(data3)
print('w3:', w3)
```

```
w1: [1 3 5 7]
w2: [2 4 6 8]
w3: [[1 2 3 4]
      [5 6 7 8]]
```

2. 专门创建数组的函数

■ arange函数：创建等差一维数组

格式： `np.arange([start, ]stop, [step, ]dtype)`

参数名称	说明
start:	起始值，默认从0开始;
stop:	结束值；生成的元素不包括结束值；
step	步长，可省略，默认步长为1；
dtype	设置元素的数据类型，默认使用输入数据的类型。

例：

```
warray=np.arange(10)  
print(warray)
```

[0 1 2 3 4 5 6 7 8 9]

```
warray=np.arange(0, 1, 0.2)  
print(warray)
```

[0. 0.2 0.4 0.6 0.8]

2. 专门创建数组的函数

■ **linspace** 函数：创建等比一维数组，接收元素数量作为参数。

格式：**np.linspace(start, stop, num, endpoint, retstep=False, dtype=None)**

参数名称	说明
start:	起始值，默认从0开始;
stop:	结束值；生成的元素不包括结束值；
num	要生成的等间隔样例数量

例：

```
warray=np.linspace(0,1,5)  
print(warray)
```

```
[0.    0.25 0.5  0.75 1.   ]
```

2. 专门创建数组的函数

■ logspace函数：创建等比一维数组

格式：`np.logspace(start, stop, num, endpoint=True, base=10.0, dtype=None)`

`logspace`的参数中，`start, stop`代表的是10的幂,默认基数`base`为10，第三个参数元素表示一维数组的个数。

例：

```
#生成1-10间的具有5个元素的等比数列  
warray=np.logspace(0, 1, 5)  
print(warray)
```

```
[ 1.          1.77827941  3.16227766  5.62341325 10.          ]
```

2. 专门创建数组的函数

- **zeros**函数：创建指定长度或形状的全0数组。

格式： `np.zeros(shape, dtype=float, order='C')`

- **ones**函数：创建指定长度或形状的全1数组。

格式： `np.ones(shape, dtype=None, order='C')`

- 使用**empty()**函数创建一个随机数组。

格式： `np.empty(shape, dtype=float, order='C')`

2. 专门创建数组的函数

■ eye()函数

eye(N, M=None, k=0)

返回值：返回一个**对角线全1，其余位置全是0**的二维数组，其中**k=0**时，**全1对角线为主对角线**；**k>0**时，**全1对角线向右上方偏移**；**k<0**时，**全1对角线向左下方偏移**。

参数说明：

N：行数；

M：可选参数，列数，默认值为**N**；

k：**k=0**时，**全1对角线为主对角线**；**k>0**时，**全1对角线向上移动相应的位置**；**k<0**时，**全1对角线向下移动相应的位置**。

2. 专门创建数组的函数

```
► import numpy as np
arr1=np.zeros(shape=(2,2))
arr2=np.ones(shape=(2,2))
arr3=np.empty(shape=(3,3))
arr4=np.eye(3)
print(arr1)
print(arr2)
print(arr3)
print(arr4)
```

```
[[0.  0.]
 [0.  0.]]
[[1.  1.]
 [1.  1.]]
[[0.00000000e+000  0.00000000e+000  0.00000000e+000]
 [0.00000000e+000  0.00000000e+000  1.05137169e-320]
 [2.18565566e-312  8.70018274e-313  0.00000000e+000]]
[[1.  0.  0.]
 [0.  1.  0.]
 [0.  0.  1.]]
```

2. 专门创建数组的函数

```
In [18]: ► arr1=np.eye(3, k = 0)  
arr2=np.eye(3, k = 1)  
print(arr1)  
print(arr2)
```

```
[[1.  0.  0.]  
 [0.  1.  0.]  
 [0.  0.  1.]]  
[[0.  1.  0.]  
 [0.  0.  1.]  
 [0.  0.  0.]]
```

2. 专门创建数组的函数

使用`identity()`函数创建数组：

`identity(n, dtype=None)`

返回值：返回 $n \times n$ 单位矩阵，主对角线上元素都为1，其他元素都为0。

参数说明：

n：单位矩阵的行数（也是列数）；

dtype：指定矩阵元素的数据类型。

```
In [16]: ► arr1=np.identity(3, dtype=int)
          print(arr1)
```

```
[[1 0 0]
 [0 1 0]
 [0 0 1]]
```

2. 专门创建数组的函数

使用full()函数创建数组

full(shape, fill_value, dtype=None, order='C')

返回值：返回由固定值填充的数组。

参数说明：

shape: int或int类型序列，表示矩阵形状；

fill_value: 填充值；

dtype: 可选参数，指定元素的数据类型；

order: 可选参数，取值'C'或者'F'，表示数组在内存的存放次序是以行(C)为主还是以列(F)为主，默认值为'C'。

```
In [17]: ▶ arr2=np.full((2, 3), 8)
          print(arr2)
```

```
[[8 8 8]
 [8 8 8]]
```

3. 使用ndarray()函数创建ndarray数组，其常用参数如下表所示：

参数	类型	作用
shape	int型tuple	指定数组的维数和每维的大小
dtype	int型、float型等	指定数组中元素的类型
buffer	ndarray	用于初始化数组的数组
offset	int	buffer 中用于初始化数组的首个数据的偏移，是数组元素在内存所占字节数的整数倍
strides	int型tuple	每个轴的下标增加1时，数据指针在内存中增加的字节数
order	'C' 或者 'F'	'C' 表示行优先， buffer 中的数据按行的顺序存入将要创建的数组中； 'F' 表示列优先

3. 使用ndarray()函数创建ndarray数组举例：

```
#buffer中的数据按行的顺序存入将要创建的数组e中  
e=np.ndarray(shape=(2,3), dtype=int, buffer=np.array([0,1,2,3,4,5,6,7,9]),  
             offset=0, order="C")  
#buffer中的数据按列的顺序存入将要创建的数组e中  
f=np.ndarray(shape=(2,3), dtype=int, buffer=np.array([0,1,2,3,4,5,6,7,9]),  
             offset=0, order="F")  
  
print(e)  
print(f)
```

```
[[0 1 2]  
 [3 4 5]]  
[[0 2 4]  
 [1 3 5]]
```

➤ ndarray对象属性和数据转换

表 ndarray对象属性及其说明

ndarray数组对象的属性	属性说明
T	ndarray.T 返回数组的转置
dtype	描述数组元素的类型
shape	返回数组的型，以元组表示的数组的各维的大小
ndim	返回数组的维度
size	ndarray.size 返回数组中元素的个数
itemsize	返回数组中的单个元素在内存所占字节数
flat	返回一个数组的迭代器，对flat赋值将导致整个数组的元素被覆盖
real/imag	给出复数数组的实部/虚部
nbytes	返回数组的所有元素占用的存储空间

➤ ndarray对象属性和数据转换

```
warray = np.array([[1, 2, 3], [4, 5, 6]])  
print(' 维度为: ', warray.ndim)  
print(' 形状为: ', warray.shape)  
print(' 元素个数为: ', warray.size)
```

维度为: 2

形状为: (2, 3)

元素个数为: 6

➤ ndarray对象属性和数据转换

【例】设置数组的shape属性。

```
warray = np.array([[1, 2, 3], [4, 5, 6]])  
warray.shape = 3, 2  
print(warray)
```

```
[[1 2]  
 [3 4]  
 [5 6]]
```

【例】数组的类型转换。

```
arr1 = np.arange(6)  
print(arr1.dtype)  
arr2 = arr1.astype(np.float64)  
print(arr2.dtype)
```

```
int32  
float64
```

➤ ndarray对象属性

```
❏ a = np.array([[ 0, 1, 2, 3, 4],[ 5, 6, 7, 8, 9],[10, 11, 12, 13, 14]])
print('数组中元素的个数为: ', a.size)
print('数组的各维的大小为: ', a.shape)
print('数组的维度为: ', a.ndim)
print('数组元素的类型为: ', a.dtype)
print('原ndarray为: \n', a)
print('转置数组为: \n', a.T)
```

数组中元素的个数为: 15

数组的各维的大小为: (3, 5)

数组的维度为: 2

数组元素的类型为: int32

原ndarray为:

```
[[ 0  1  2  3  4]
 [ 5  6  7  8  9]
 [10 11 12 13 14]]
```

转置数组为:

```
[[ 0  5 10]
 [ 1  6 11]
 [ 2  7 12]
 [ 3  8 13]
 [ 4  9 14]]
```

➤ ndarray对象属性

```
In [23]: ▶ print(a.flat) #返回数组的迭代器  
for x in a.flat:  
    print(x, end=',')
```

```
<numpy.flatiter object at 0x00000234C1F0B700>  
0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14,
```

➤ 简单随机数

numpy.random模块中常用的随机分布函数如所示

随机分布函数	函数功能
binomial(n, p, size=None)	产生size个二项分布的样本值，n表示n次试验，p表示每次实验发生（成功）的概率。函数的返回值表示n次实验中发生（成功）的次数
exponential(scale=1.0, size=None)	产生size个指数分布的样本值，这里的scale是 β ，为标准差， $\beta=1/\lambda$ ， λ 为单位时间内事件发生的次数
normal(loc=0.0, scale=1.0, size=None)	产生size个正态（高斯）分布的样本值，loc为正态分布的均值，scale为正态分布的标准差，size缺省，则采样一个
poisson(lam=1.0, size=None)	从泊松分布中生成随机数，lam是单位时间内事件的平均发生次数
uniform(low=0.0, high=1.0, size=None)	产生size个均匀分布[low,high)的样本值，size为int或元组类型，如size = (m, n, k)，则输出m*n*k个样本，缺省时输出1个值

➤ 简单随机数

(1) `randint()`函数：在NumPy.random模块中，提供了多种随机数的生成函数。如`randint`函数生成指定范围的随机整数来构成指定形状的数组。

用法：**`np.random.randint(low, high = None, size = None)`**
生成size个随机整数，取值区间为[low, high)，若没有输入参数high，则取值区间为[0,low)

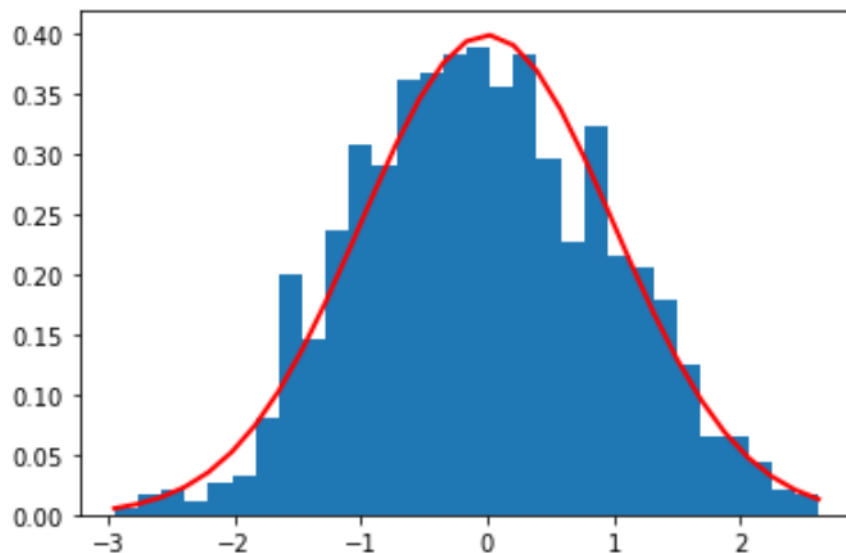
```
arr = np.random.randint(100, 200, size = (2, 4))  
print(arr)
```

```
[[112 173 165 106]  
 [145 156 180 139]]
```

➤ 随机分布

(2) `normal(loc=0.0, scale=1.0, size=None)`: 产生size个正态（高斯）分布的样本值。

```
from numpy import random
import numpy as np
import matplotlib.pyplot as plt
mu, sigma = 0, 1
s = random.normal(loc=mu, scale=sigma, size=1000) #获取1000个样本值
#绘制样本的直方图, 以及概率密度函数
count, bins, patches = plt.hist(s, 30, density=True)
plt.plot(bins, 1/(sigma * np.sqrt(2 * np.pi)) * np.exp(-(bins - mu)**2/(2 * sigma**2)),
         linewidth=2, color='r')
plt.show() #绘制的直方图以及概率密度函数曲线
```



➤ 随机排序

numpy.random模块中用于对数组对象随机排列的函数

函数名称	函数功能
shuffle(x)	打乱对象x（多维数组按照第一维打乱），直接在原来的数组上进行操作，改变原来数组元素的顺序，无返回值，x可以是数组或者列表
permutation(x)	打乱并返回新对象（多维数组按照第一维打乱），不直接在原数组上进行操作，而是返回一个新的打乱元素顺序的数组，并不改变原来的数组，x可以是整数或者列表，如果是整数k，那就随机打乱 numpy.arange(k)

➤ 随机排序

numpy.random模块中用于对数组对象随机排列的函数

shuffle(x): 打乱对象x（多维数组按照第一维打乱），直接在原来的数组上进行操作，改变原来数组元素的顺序，无返回值，x可以是数组或者列表。

```
import numpy as np
arr1=np.arange(9).reshape((3, 3))
print('原数组: \n', arr1)
np.random.shuffle(arr1)
print('随机排序后的数组: \n', arr1)
```

原数组:

```
[[0 1 2]
 [3 4 5]
 [6 7 8]]
```

随机排序后的数组:

```
[[0 1 2]
 [6 7 8]
 [3 4 5]]
```

➤ 数组变换

ndarray数组对象提供了一些方法用来改变数组的形状，常用的改变数组形状的方法如表所示。

改变数组形状的方法	功能
ndarray.reshape(shape,order)	返回一个具有相同数据域，但shape不一样的视图
ndarray.resize(new_shape,overwrite)	原地修改数组的形状，需要保持元素个数前后相同
ndarray.transpose(*axes)	返回数组针对某一轴进行转置后的数组，对于二维ndarray，transpose在不指定参数时默认是矩阵转置
ndarray.swapaxes(axis1,axis2)	返回数组axis1轴与axis2轴互换的视图
ndarray.flatten(order)	返回将原数组展平后的一维数组的拷贝（全新的数组）
ndarray.ravel(order='C')	返回将原数组展平后的一维数组的视图， order='C' 表示行序优先， order='F' 表示列序优先

➤ 数组变换

1. 数组重塑(reshape函数)

对于定义好的数组，可以通过reshape方法改变其数据维度。

格式：`np.reshape(a, newshape, order='C')`

参数名称	说明
a	需要处理的数据。
newshape	新维度——整数或整数元组。

➤ 数组变换

1. 数组重塑(reshape函数)

例:

```
arr1=np.arange(8)
print('原数组: \n', arr1)
arr2=arr1.reshape(4, 2)
print('重塑后的数组: \n', arr2)
```

原数组:

[0 1 2 3 4 5 6 7]

重塑后的数组:

[[0 1]

[2 3]

[4 5]

[6 7]]

➤ 数组变换

1. 数组重塑(reshape函数)

- reshape的参数中的其中一个可以设置为-1，表示数组的维度可以通过数据本身来推断。

```
arr1=np.arange(12)
print('arr1:', arr1)
arr2=arr1.reshape(2, -1)
print('arr2:\n', arr2)
```

```
arr1: [ 0  1  2  3  4  5  6  7  8  9 10 11]
arr2:
[[ 0  1  2  3  4  5]
 [ 6  7  8  9 10 11]]
```

➤ 数组变换

2. 数组重塑（ravel函数和flatten函数）

- 与reshape相反的方法是数据散开（ravel）或数据扁平化（flatten），数据重塑不会改变原来的数组。

```
arr1=np.arange(12).reshape(3,4)
arr2=arr1.ravel()
arr3=arr1.flatten()
print('arr1:\n',arr1)
print('arr2:',arr2)
print('arr3:',arr3)
```

```
arr1:
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
arr2: [ 0  1  2  3  4  5  6  7  8  9 10 11]
arr3: [ 0  1  2  3  4  5  6  7  8  9 10 11]
```

➤ 数组变换

3. 数组合并（**hstack**函数、**vstack**函数和**concatenate**函数）

hstack函数：实现横向合并；

vstack函数：实现纵向组合是利用**vstack**将数组纵向合并；

concatenate函数：可以实现数组的横向或纵向合并，参数**axis=1**时进行横向合并，**axis=0**时进行纵向合并。



➤ 数组变换

3. 数组合并（**hstack**函数、**vstack**函数和**concatenate**函数）

```
arr1=np.arange(6).reshape(3,2)
arr2=arr1*2
arr3=np.vstack((arr1,arr2))
arr4=np.hstack((arr1,arr2))
print('arr1:\n',arr1)
print('arr2:\n',arr2)
print(' 横向合并hstack为: \n',arr4)
print(' 纵向合并vstack为: \n',arr3)
```

```
arr1:
[[0 1]
 [2 3]
 [4 5]]
arr2:
[[ 0  2]
 [ 4  6]
 [ 8 10]]
横向合并hstack为:
[[ 0  1  0  2]
 [ 2  3  4  6]
 [ 4  5  8 10]]
纵向合并vstack为:
[[ 0  1]
 [ 2  3]
 [ 4  5]
 [ 0  2]
 [ 4  6]
 [ 8 10]]
```

➤ 数组变换

4. 数组分割 (hsplit函数、vsplit函数和split函数)

与数组合并相反，hsplit函数、vsplit函数和split函数分别实现数组的横向、纵向和指定方向的分割。

```
arr = np.arange(16).reshape(4,4)
arr1=np.hsplit(arr,2)
arr2=np.vsplit(arr,2)
print(arr)
print(' 横向分割为:\n',arr1)
print(' 纵向分割为:\n',arr2)
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]]
```

横向分割为:

```
[array([[ 0,  1],
        [ 4,  5],
        [ 8,  9],
        [12, 13]]), array([[ 2,  3],
        [ 6,  7],
        [10, 11],
        [14, 15]])]
```

纵向分割为:

```
[array([[0, 1, 2, 3],
        [4, 5, 6, 7]]), array([[ 8,  9, 10, 11],
        [12, 13, 14, 15]])]
```

➤ 数组变换

5. 数组转置和轴对换

数组转置是数组重塑的一种特殊形式，可以通过**transpose**方法进行转置。除了使用**transpose**外，可以直接利用数组的**T**属性进行数组转置。

```
arr=np.arange(6).reshape(3,2)
arr1=arr.transpose((1,0))
arr2=arr.T
print('原矩阵:\n',arr)
print('函数转置矩阵:\n',arr1)
print('属性转置矩阵:\n',arr2)
```

原矩阵:

```
[[0 1]
 [2 3]
 [4 5]]
```

函数转置矩阵:

```
[[0 2 4]
 [1 3 5]]
```

属性转置矩阵:

```
[[0 2 4]
 [1 3 5]]
```

➤ 索引和切片

- 数组索引机制指的是用方括号[]加序号的形式抽取元素、选取数组的某些元素、以及为索引处的元素重新赋值。
- 数组的切片操作是用来抽取数组的一部分元素生成新数组。切片是把用冒号“:”隔开的数字置于方括号“[]”里。
- 需要注意的是：对Python列表进行切片操作得到的列表是原列表的拷贝，而NumPy对ndarray数组进行切片操作得到的数组是指向相同缓冲区的视图，对所得切片数组元素的改变就是对原数组元素的改变。

➤ 一维数组的索引和切片

- 一维数组的索引类似Python中的列表。

```
arr=np.arange(10)
print(arr)
print(arr[2])
print(arr[-1])
print(arr[1:4])
```

```
[0 1 2 3 4 5 6 7 8 9]
```

```
2
```

```
9
```

```
[1 2 3]
```

➤ 一维数组的索引和切片

- 数组的切片返回的是原始数组的视图，不会产生新的数据，如果需要的并非视图而是要复制数据，则可以通过copy方法实现。

```
arr=np.arange(10)
arr1=arr[-4:-1].copy()
print(arr)
print(arr1)
```

```
[0 1 2 3 4 5 6 7 8 9]
[6 7 8]
```

➤ 索引和切片

2. 使用列表索引数组

```
x = np.arange(10, 1, -1)
print(x)
x[[2, 2, 1, 6]]
```

```
[10  9  8  7  6  5  4  3  2]
```

```
array([8, 8, 9, 4])
```

3. 布尔值索引数组

```
y = np.arange(30)
b = y > 20
y[b]
```

```
array([21, 22, 23, 24, 25, 26, 27, 28, 29])
```

➤ 多维数组的索引和切片

- 对于多维数组，它的每一个维度都有一个索引，各个维度的索引之间用逗号分隔。
- 也可以使用整数函数和布尔值索引访问多维数组。

```
arr = np.arange(12).reshape(3, 4)
print(arr)
#索引第0行中第1列到第2列的元素
print('第0行中第1列到第2列的元素:\n', arr[0, 1:3])
print('第2列元素\n', arr[:, 2])    #索引第2列元素
print('第0行第0列元素\n', arr[:1, :1]) #第0行第0列元素
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

第0行中第1列到第2列的元素:

```
[1 2]
```

第2列元素

```
[ 2  6 10]
```

第0行第0列元素

```
[[0]]
```

➤ 多维数组的索引和切片

- 对于二维数组，当行、列索引都有冒号时，不降维，结果仍是一个二维数组。
- 当行、列索引只有一个冒号时，降维，结果是一个一维数组。
- 当行、列索引都无冒号时，结果是一个元素。

➤ 多维数组的索引和切片

```
arr = np.arange(12).reshape(3, 4)
#从两个序列的对应位置取出两个整数来组成下标: arr[0, 1], arr[1, 3]
print(arr)
print('索引结果1: ', arr[(0, 1), (1, 3)])
#索引第1、2行中第0、2、3列的元素
print('索引结果2: ', arr[1:2, (0, 2, 3)])
mask = np.array([1, 0, 1], dtype = np.bool)
#mask是一个布尔数组, 它索引第0, 2行中第1列元素
print('索引结果3: ', arr[mask, 1])
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

索引结果1: [1 7]

索引结果2: [[4 6 7]]

索引结果3: [1 9]

➤ 选择数组元素的方法

(1) `ndarray.take(indices[,axis=None,out=None, mode='raise'])`

根据指定的索引`indices`从数组对象`ndarray`中获取对应元素，并构成一个新的数组返回。

axis: 用来指定选择元素的轴，默认情况下，使用扁平化的输入数组，即把数组当成一个一维数组；

out: 是`ndarray`对象，用来存放函数返回值，要求其`shape`必须与函数返回值的`shape`一致；

mode: 指定越界索引将如何处理

➤ 选择数组元素的方法

(1) `ndarray.take(indices[,axis=None,out=None, mode='raise'])`

```
x=np.array([ 0,  2,  4,  6,  8, 10, 12, 14, 16, 18])  
print('第1次取出的元素: \n', x.take([0, 2, 4]))  
print('第2次取出的元素: \n', x.take([[2, 5], [3, 6]]))
```

第1次取出的元素:

[0 4 8]

第2次取出的元素:

[[4 10]

[6 12]]

➤ 选择数组元素的方法

(2) `ndarray.put(indices, values[, mode])`

将数组中索引`indices`指定的位置处的元素值设置为`values`中对应的元素值。

```
x = np.arange(0, 20, 2)
print(x)
#将x中索引[0, 1]处的值设置为列表[1, 3]中对应的值
x.put([0, 1], [1, 3])
print(x)
```

```
[ 0  2  4  6  8 10 12 14 16 18]
[ 1  3  4  6  8 10 12 14 16 18]
```

➤ 选择数组元素的方法

(3) `ndarray.searchsorted(v, side='left', sorter=None)`

将`v`插入到当前有序的数组中，返回插入的位置索引。

```
▶ w = np.array([1, 2, 3, 3, 3, 3, 6, 7, 9, 10, 12])  
w.searchsorted(3)  
print(w.searchsorted(3))  
print(w.searchsorted(3, side='right'))           # side='right'
```

2

6

➤ 选择数组元素的方法

(4) `ndarray.itemset(*args)`: 修改数组中某个元素的值。

```
a = np.array([[0, 1, 2], [33, 4, 5], [6, 7, 8]])  
print('修改前: \n', a)  
a.itemset((1, 2), 12)  
print('修改后: \n', a)
```

修改前:

```
[[ 0  1  2]  
 [33  4  5]  
 [ 6  7  8]]
```

修改后:

```
[[ 0  1  2]  
 [33  4 12]  
 [ 6  7  8]]
```

➤ ufunc函数

- **ufunc**函数全称为通用函数，是一种能够对数组中的**所有元素进行操作的函数**。对一个数组进行重复运算时，使用**ufunc**函数比使用**math**库中的函数效率要高很多。

1. 常用的ufunc函数运算

常用的ufunc函数运算有四则运算、比较运算和逻辑运算。

1) 四则运算：

加 (+)、减 (-)、乘 (*)、除 (/)、幂 (**)。数组间的四则运算表示对每个数组中的元素分别进行四则运算，所以形状必须相同。

2) 比较运算：

>、<、==、>=、<=、!=。比较运算返回的结果是一个布尔数组，每个元素为每个数组对应元素的比较结果。

➤ 算术运算与函数运算

numpy模块中对两个数组的元素进行算术运算的二元函数。

函数	说明
<code>add(a,b)</code>	将两个数组中对应的元素相加， <code>a</code> 、 <code>b</code> 为两个 <code>ndarray</code> 数组
<code>subtract(a,b)</code>	将两个数组中对应的元素相减
<code>multiply(a,b)</code>	两个数组中对应的元素相乘
<code>power(a,b)</code>	对第一个数组中的元素 <code>x</code> ，第二个数组中的对应 位置的元素 <code>y</code> ，计算 <code>x</code> 的 <code>y</code> 次方
<code>greater</code> 、 <code>greater_equal</code> 、 <code>less</code> 、 <code>less_equal</code> 、 <code>equal</code> 、 <code>not_equal</code>	将两个数组中对应的元素进行比较运算，最终 产生布尔型的数组。相当于运算符 <code>></code> 、 <code>>=</code> 、 <code><</code> 、 <code><=</code> 、 <code>==</code> 、 <code>!=</code>

➤ 算术运算与函数运算

数组最常用的运算是算术运算，可以为数组的每个元素加上或乘以某个数值，可以让两个数组对应元素之间做加、减、乘、除运算等。

```
import numpy as np
a=np.array([[1,2,3],[4,5,6]])
print('a=\n',a)
print('a+6=\n',a+6)
```

```
a=
[[1 2 3]
 [4 5 6]]
a+6=
[[ 7  8  9]
 [10 11 12]]
```

➤ 算术运算与函数运算

通过在数组和数字之间使用条件运算符，比如大于号，将会得到由布尔值组成的数组，对于原数组中满足条件的元素，布尔数组中处于同等位置的元素为True。

```
import numpy as np
a=np.array([[0.51691317, 0.20933602, 0.22460165],
            [0.91598144, 0.43223077, 0.59339314]])
b=a[a>0.5]
print('a>0.5?\n',a>0.5)
print('a>0.5:\n',b)
```

a>0.5?

```
[[ True False False]
 [ True False  True]]
```

a>0.5:

```
[0.51691317 0.91598144 0.59339314]
```

➤ 算术运算与函数运算

逻辑运算：

np.any函数表示逻辑“or”，np.all函数表示逻辑“and”，运算结果返回布尔值。

```
a = np.array([1, -1, 5])  
b = np.array([4, 0, 3])  
c = np.any([a, b])  
print('a=', a)  
print('b=', b)  
print('any(a, b)=', c)
```

```
a= [ 1 -1  5]  
b= [4 0 3]  
any(a, b)= True
```

➤ 条件逻辑运算

在NumPy中可以利用基本的逻辑运算就可以实现数组的条件运算。

```
arr1 = np.array([1, 3, 5, 7])
arr2 = np.array([2, 4, 6, 8])
cond = np.array([True, False, True, False])
result = [(x if c else y) for x, y, c in zip(arr1, arr2, cond)]
result
```

[1, 4, 5, 8]

这种方法对大规模数组处理效率不高，也无法用于多维数组。NumPy提供的where方法可以克服这些问题。

➤ 条件逻辑运算

where的用法:

np.where(condition, x, y)

满足条件(**condition**), 输出**x**, 不满足则输出**y**。

```
np.where([[True, False], [True, True]], [[1, 2], [3, 4]], [[9, 8], [7, 6]])  
  
array([[1, 8],  
       [3, 4]])
```

条件为[[True,False], [True,False]], 分别对应最后输出结果的四个值, 运算时第一个值从[1,9]中选, 因为条件为True, 所以是选1。第二个值从[2,8]中选, 因为条件为False, 所以选8, 后面以此类推。

➤ 条件逻辑运算

【例3-35】 `where`中只有`condition`参数。

```
w = np.array([2, 5, 6, 3, 10])  
np.where(w>4)  
  
(array([1, 2, 4], dtype=int64),)
```

`where`中若只有条件 (`condition`)，没有`x`和`y`，则输出满足条件元素的坐标。这里的坐标以`tuple`的形式给出，通常原数组有多少维，输出的`tuple`中就包含几个数组，分别对应符合条件元素各维坐标。

➤ 算术运算与函数运算

【例】数组的比较运算。

```
x = np.array([1, 3, 6])
y = np.array([2, 3, 4])
print('比较结果(<): ', x<y)
print('比较结果(>): ', x>y)
print('比较结果(==): ', x==y)
print('比较结果(>=): ', x>=y)
print('比较结果(!=): ', x!=y)
```

```
比较结果(<):  [ True False False]
比较结果(>):  [False False  True]
比较结果(==): [False  True False]
比较结果(>=): [False  True  True]
比较结果(!=): [ True False  True]
```

➤ 算术运算与函数运算

ufunc函数的广播机制

广播（broadcasting）是指不同形状的数组之间执行算术运算的方式。需要遵循4个原则：

- 1) 让所有输入数组都向其中`shape`最长的数组看齐，`shape`中不足的部分都通过在左边加1补齐。
- 2) 如果两个数组的形状在任何一个维度上都不匹配，那么数组的形状会沿着维度为1的维度进行扩展，以匹配另一个数组的形状。

➤ 算术运算与函数运算

广播（broadcasting）是指不同形状的数组之间执行算术运算的方式。需要遵循4个原则：

3) 输出数组的shape是输入数组shape的各个轴上的最大值。

4) 如果两个数组的形状在任何一个维度上都不匹配，并且没有任何一个维度等于1，则引发异常。

➤ 算术运算与函数运算

ufunc函数的广播机制

【例】ufunc函数的广播

```
arr1 = np.array([[0, 0, 0], [1, 1, 1], [2, 2, 2]])  
arr2 = np.array([1, 2, 3])  
print('arr1:\n', arr1)  
print('arr2:\n', arr2)  
print('arr1+arr2:\n', arr1+arr2)
```

```
arr1:  
[[0 0 0]  
 [1 1 1]  
 [2 2 2]]  
arr2:  
[1 2 3]  
arr1+arr2:  
[[1 2 3]  
 [2 3 4]  
 [3 4 5]]
```

➤ 常用统计函数

NumPy中提供了很多用于统计分析的函数，常见的有**sum**、**mean**、**std**、**var**、**min**和**max**等。

ndarray数组对象的很多统计计算方法都有一个**axis**参数，它有如下作用：

1. 当**axis=None**（默认）时，数组被当成一个**一维数组**，对数组的计算操作是**对整个数组**进行的，比如**sum**方法，就是求数组中所有元素的和；
2. 当**axis**被指定为一个**int**整数时，对数组的计算操作是以提供的**axis**轴进行的，**axis=0**表示对**column**（**列**）进行操作，**axis=1**表示对**row**（**行**）进行操作。

➤ 常用统计函数

ndarray数组对象的常用统计计算方法	功能
<code>ndarray.max(axis=None, out=None)</code>	返回根据指定的axis计算最大值，axis=0表示求各column的最大值，axis=1表示求各row的最大值
<code>ndarray.argmax(axis=None, out)</code>	返回根据指定axis计算最大值的索引，out是ndarray对象，用来存放函数返回值，要求其shape必须与函数返回值的shape一致
<code>ndarray.min(axis=None, out=None)</code>	返回根据指定的axis计算最小值
<code>ndarray.argmin(axis=None, out=None)</code>	返回指定axis最小元素的索引
<code>ndarray.ptp(axis, out)</code>	返回根据指定axis计算最大值与最小值的差
<code>ndarray.clip(min, max, out)</code>	返回数组元素限制在[min, max]之间的新数组，小于min的转为min，大于max的转为max

➤ 常用统计函数

<code>ndarray.trace(offset=0, axis1=0, axis2=1, dtype=None, out=None)</code>	返回数组的迹（对角线元素的和）， <code>offset</code> 表示离开主对角线的偏移量
<code>ndarray.sum(axis=None, dtype=None, out=None)</code>	返回指定 <code>axis</code> 的所有元素的和，默认求所有元素的和
<code>ndarray.cumsum(axis=None, dtype=None, out=None)</code>	按照所给定的轴参数返回元素的累计和
<code>ndarray.mean(axis=None, dtype=None, out=None)</code>	返回指定 <code>axis</code> 的数组元素均值
<code>ndarray.var(axis=None, dtype=None, out=None, ddof=0)</code>	根据指定的 <code>axis</code> 计算数组的方差
<code>ndarray.std(axis=None, dtype=None, out=None, ddof=0)</code>	根据指定 <code>axis</code> 计算数组的标准差
<code>ndarray.prod(axis=None, dtype=None, out=None)</code>	返回指定轴的所有元素乘积
<code>ndarray.cumprod(axis=None, dtype=None, out=None)</code>	返回指定轴的累积

➤ 统计计算

(1) `ndarray.max(axis=None, out=None)`: 返回根据指定的axis计算最大值, `axis=0`表示求各column的最大值, `axis=1`表示求各row的最大值, `out`是`ndarray`对象, 用来存放函数返回值, 要求其`shape`必须与函数返回值的`shape`一致。

```
import numpy as np
a=np.array([[2, 3, 4, 9], [8, 7, 6, 5], [4, 3, 5, 8]])
o=np.ndarray(shape=3)
# axis=1表示求各row的最大值
a.max(axis=1, out=o)
print(o)
```

```
[9. 8. 8.]
```

➤ 统计计算

(2) `ndarray.clip(min, max, out)`: 返回数组元素限制在`[min, max]`之间的新数组（小于`min`的转为`min`，大于`max`的转为`max`）。

```
▶ #返回数组元素限制在[5, 8]之间的新数组，小于5的转为5，大于8的转为8  
a=np.array([[2, 3, 4, 9], [8, 7, 6, 5], [4, 3, 5, 8]])  
print(a.clip(5, 8))
```

```
[[5 5 5 8]  
 [8 7 6 5]  
 [5 5 5 8]]
```

➤ 统计计算

(3) `ndarray.cumsum(axis=None, dtype=None, out=None)`: 按照所给定的轴参数返回元素的累计和。

```
a=np.array([[2, 3, 4, 9], [8, 7, 6, 5], [4, 3, 5, 8]])  
print('a=\n', a)  
#按行求累计和  
print('按行求累计和=\n', a.cumsum(axis=1))
```

```
a=  
[[2 3 4 9]  
 [8 7 6 5]  
 [4 3 5 8]]  
按行求累计和=  
[[ 2  5  9 18]  
 [ 8 15 21 26]  
 [ 4  7 12 20]]
```

➤ 重复数据与去重

(4) 统计分析中有时也需要把一个数据重复若干次，使用**tile**和**repeat**函数即可实现此功能。

◆ **tile**函数的格式: `np.tile(A, reps)`

其中，参数**A**表示要重复的数组，**reps**表示重复次数。

◆ **repeat**函数的格式: `np.repeat(A, reps, axis = None)`

“a”: 是需要重复的数组元素，

“repeats”: 是重复次数，

“axis”: 指定沿着哪个轴进行重复，**axis = 0**表示按列进行元素重复；**axis = 1**表示按行进行元素重复。

➤ 重复数据与去重

【例】使用tile函数实现数据重复

```
arr = np.arange(5)
print('原数组: ', arr)
wy = np.tile(arr, 3)
print('重复数据: \n', wy)
```

原数组: [0 1 2 3 4]

重复数据:

[0 1 2 3 4 0 1 2 3 4 0 1 2 3 4]

➤ 重复数据与去重

【例】使用repeat函数实现数据重复

```
arr = np.arange(5)
print('原数组: ', arr)
wy = np.tile(arr, 3)
print('重复数据1: \n', wy)
arr2 = np.array([[1, 2, 3], [4, 5, 6]])
print('重复数据2: \n', arr2.repeat(2, axis=0))
```

原数组: [0 1 2 3 4]

重复数据1:

[0 1 2 3 4 0 1 2 3 4 0 1 2 3 4]

重复数据2:

[[1 2 3]

[1 2 3]

[4 5 6]

[4 5 6]]

➤ 重复数据与去重

(5) 在NumPy中，对于一维数组或者列表，**unique**函数去除其中重复的元素，并按元素由大到小返回一个新的元组或者列表。

```
names = np.array(['红色', '蓝色', '黄色', '白色', '红色'])  
print('原数组:', names)  
print('去重后的数组:', np.unique(names))
```

原数组: ['红色' '蓝色' '黄色' '白色' '红色']

去重后的数组: ['白色' '红色' '蓝色' '黄色']

➤ 常用统计函数

```
arr = np.arange(20).reshape(4, 5)
print('创建的数组: \n', arr)
print('数组的和: ', np.sum(arr))
print('数组纵轴的和: ', np.sum(arr, axis = 0))
print('数组横轴的和: ', np.sum(arr, axis = 1))
print('数组的均值: ', np.mean(arr))
print('数组横轴的均值: ', np.mean(arr, axis = 1))
print('数组的标准差: ', np.std(arr))
print('数组横轴的标准差: ', np.std(arr, axis = 1))
```

创建的数组:

```
[[ 0  1  2  3  4]
 [ 5  6  7  8  9]
 [10 11 12 13 14]
 [15 16 17 18 19]]
```

数组的和: 190

数组纵轴的和: [30 34 38 42 46]

数组横轴的和: [10 35 60 85]

数组的均值: 9.5

数组横轴的均值: [2. 7. 12. 17.]

数组的标准差: 5.766281297335398

数组横轴的标准差: [1.41421356 1.41421356 1.41421356 1.41421356]

➤ 排序

ndarray数组对象的排序元素的常用方法如表所示。

排序元素的方法	方法功能
ndarray.sort(axis=-1, kind='quicksort', order=None)	原地对数组元素进行排序，即排序后改变了原数组， axis 指定排序沿着数组的（轴）方向，0表示按列，1表示按行， axis 默认值为-1，表示沿最后的轴排序； kind 指定排序的算法，其取值集合为{'quicksort', 'mergesort', 'heapsort'}， order 指定元素的排列顺序，默认升序
ndarray.argsort(axis=-1, kind='quicksort', order=None)	返回对数组进行升序排序之后的数组元素在原数组中的索引

➤ 排序

Sort函数对数据直接进行排序，调用改变原始数组，无返回值。

格式：**numpy.sort(a, axis, kind, order)**

主要参数及其说明见表3-5。

参数	使用说明
a	要排序的数组
kind	排序算法，默认为“quicksort”
order	排序的字段名，可指定字段排序，默认为None
axis	使得 sort 函数可以沿着指定轴对数据集进行排序。 axis=1 为沿横轴排序； axis=0 为沿纵轴排序； axis=None ，将数组平坦化之后进行排序

NumPy多维数组的运算



➤ 排序

(1) `ndarray.sort(axis=-1, kind='quicksort', order=None)`: 原地对数组元素进行排序。

```
y1=np.array([[0, 15, 10, 5], [25, 22, 3, 2], [55, 45, 59, 50]])  
print('原y1=\n', y1)  
y1.sort()  
print('排序后的y1=\n', y1)
```

原y1=

```
[[ 0 15 10  5]  
 [25 22  3  2]  
 [55 45 59 50]]
```

排序后的y1=

```
[[ 0  5 10 15]  
 [ 2  3 22 25]  
 [45 50 55 59]]
```

➤ 排序

【例】带轴向参数的sort排序

```
arr = np.array([[4, 2, 9, 5], [6, 4, 8, 3], [1, 6, 2, 4]])  
print('原数组: \n', arr)  
arr.sort(axis = 1)    #沿横向排序  
print('横向排序后: \n', arr)
```

原数组:

[[4 2 9 5]

[6 4 8 3]

[1 6 2 4]]

横向排序后:

[[2 4 5 9]

[3 4 6 8]

[1 2 4 6]]

➤ 排序

(2) **np.argsort**函数和**np.lexsort**函数根据一个或多个键值对数据集进行排序。

np.argsort(): 返回的是数组值从小到大的索引值;

np.lexsort(): 返回值是按照最后一个传入数据排序的结果。

使用**argsort**和**lexsort**函数，可以在给定一个或多个键时，得到一个由整数构成的索引数组，索引值表示数据在新的序列中的位置。

➤ 排序

【例】使用argsort函数进行排序

```
arr = np.array([7, 9, 5, 2, 9, 4, 3, 1, 4, 3])  
print('原数组: ', arr)  
#返回值为数组排序后的下标排列  
print('排序后: ', arr.argsort())
```

原数组: [7 9 5 2 9 4 3 1 4 3]

排序后: [7 3 6 9 5 8 2 0 1 4]

➤ 排序

【例】使用lexsort排序

```
a = np.array([7, 2, 1, 4])
b = np.array([5, 2, 6, 7])
c = np.array([5, 2, 4, 6])
d = np.lexsort((a, b, c))
print('排序后: ', list(zip(a[d], b[d], c[d])))
```

排序后: [(2, 2, 2), (1, 6, 4), (7, 5, 5), (4, 7, 6)]

➤ 数组拼接与切分

有时候需要用已有的数组创建新数组，有时候需要切分已有数组创建新数组。`numpy`提供了一些函数来实现数组的拼接与切分。

1) 垂直拼接

`vstack(tup)`用来将列数相同的数组序列`tup`中的数组进行竖直方向的拼接，即把数组序列`tup`中后一个数组作为行追加到前一个数组的下边，数组向竖直方向上生长。参数`tup`的类型可以是元组，列表，或者`numpy`数组，返回结果为`numpy`数组。

```
a = np.array([1, 2, 3])  
b = np.array([2, 3, 4])  
print(np.vstack((a, b)))
```

```
[[1 2 3]  
 [2 3 4]]
```

➤ 数组拼接与切分

2) 水平拼接

hstack(tup)用来将行数相同的数组序列**tup**中的数组进行水平方向的拼接，即把数组序列**tup**中后一个数组作为列追加到前一个数组的右边，数组向水平方向上生长。参数**tup**的类型可以是元组，列表，或者**numpy**数组，返回结果为**numpy**的数组。

➤ 数组拼接与切分

2) 水平拼接

```
a = np.array([1, 2, 3]).reshape(3, 1)
b = np.array([4, 5, 6]).reshape(3, 1)
c = np.hstack((a, b))
print('a=\n', a)
print('b=\n', b)
print('c=\n', c)
```

```
a=
[[1]
 [2]
 [3]]
b=
[[4]
 [5]
 [6]]
c=
[[1 4]
 [2 5]
 [3 6]]
```

➤ 数组拼接与切分

1) 水平切分

numpy提供了函数**hsplit()**用来水平方向上切分数组，水平切分是把数组按照宽度切分为两部分，例如把 4×4 的数组切分为两个 4×2 的数组。

hsplit(ary, indices_or_sections)

ary: 表示待切分的数组

indices_or_sections: 可以是一个整数，表示平均切分的个数，必须要均等分，否则会报错，也可是一个数组元素递增的整数一维数组，如[2,3]表示要将**ary**分成**ary[:2]**、**ary[2:3]**、**ary[3:]**三个子数组。

```
>>> a
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11],
       [12, 13, 14, 15]])
```

➤ 数组拼接与切分

1) 水平切分

```
a = np.arange(16).reshape(4, 4)
x, y, z = np.hsplit(a, [2, 3])
print('a:\n', a)
print('x:\n', x)
print('y:\n', y)
print('z:\n', z)
```

```
a:
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]]
```

```
x:
[[ 0  1]
 [ 4  5]
 [ 8  9]
 [12 13]]
```

```
y:
[[ 2]
 [ 6]
 [10]
 [14]]
```

```
z:
[[ 3]
 [ 7]
 [11]
 [15]]
```

➤ 数组拼接与切分

2) 竖直切分

numpy提供了函数`vsplit(ary, indices_or_sections)`用来竖直方向上切分数组。

```
a = np.arange(16).reshape(4, 4)
x, y = np.vsplit(a, 2)
print('a:\n', a)
print('x:\n', x)
print('y:\n', y)
```

```
a:
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]]
```

```
x:
[[0 1 2 3]
 [4 5 6 7]]
```

```
y:
[[ 8  9 10 11]
 [12 13 14 15]]
```

➤ 线性代数运算

在NumPy中，用“*”进行两个数组相乘是两个数组对应位置上的元素相乘。NumPy用dot()函数执行一般意义上的矩阵（用ndarray数组表示）乘积，矩阵A、B乘积运算时要求矩阵A的列数必须等于矩阵B的行数，A、B乘积结果是一个矩阵，其第m行第n列的元素等于矩阵A的第m行的元素与矩阵B的第n列对应元素乘积之和。

➤ 线性代数运算

```
A=np.arange(1,10).reshape((3,3))
B=np.ones(shape=(3,3),dtype=int)
print('A=',A)
print('B=',B)
C=np.dot(A,B)
D=A.dot(B)
E=A*B
print('dot(A,B)=',C)
print('A.dot(B)=',D)
print('A*B=',E)
```

```
A= [[1 2 3]
     [4 5 6]
     [7 8 9]]
B= [[1 1 1]
     [1 1 1]
     [1 1 1]]
dot(A,B)= [[ 6  6  6]
            [15 15 15]
            [24 24 24]]
A.dot(B)= [[ 6  6  6]
            [15 15 15]
            [24 24 24]]
A*B= [[1 2 3]
       [4 5 6]
       [7 8 9]]
```

➤ 线性代数运算

numpy的linalg模块提供了一些进行线性代数运算的函数，如表所示

函数	描述
<code>det</code>	计算矩阵行列式
<code>eig(A)</code>	计算方阵A的特征值和特征向量
<code>inv(A)</code>	计算方阵A的逆
<code>svd (A, full_matrices=1, compute_uv=1)</code>	对矩阵进行奇异值分解，该函数返回3个矩阵——U、Sigma和V，其中U和V是正交矩阵，Sigma是输入矩阵的奇异值
<code>solve(a, b)</code>	求解形如 $AX = B$ 的线性方程组，其中A是一个 $N*N$ 的二维数组，而B是一个长度为N的一维数组，数组X是待求解的线性方程组的解

➤ 创建numpy矩阵

1. 创建矩阵有两种方法:

- 使用`mat()`函数
- 使用`matrix()`函数

```
matr1 = np.mat("1, 2, 3;4, 5, 6;7, 8, 9", dtype = np.float64)
matr2 = np.matrix([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(matr1)
print(matr2)
```

```
[[1.  2.  3.]
 [4.  5.  6.]
 [7.  8.  9.]]
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

2. 矩阵运算：

- 矩阵运算是针对整个矩阵中的每个元素进行的，和使用循环相比，在运算速度上更快。
- 矩阵和矩阵之间可以进行+、-、*、/，其中*为点乘，需要特别注意。如果要对应元素相乘可以使用`multiply()`函数。
- 除了能够实现算术运算外，矩阵还有特有的一些属性。

属性	含义
T	返回矩阵的转置
H	返回矩阵的共轭转置
I	返回逆矩阵
A	返回矩阵的视图

➤ 读写二进制文件

numpy中的**save()**函数以二进制格式保存数组到一个文件中，文件的扩展名为“**.npz**”，该扩展名是由系统自动添加的。

numpy中的**load()**函数从二进制文件中读取数据。**save()**函数的语法格式如下：

numpy.save(file, arr)

参数说明：

file：用来保存数组的文件名或文件路径，是字符串类型。

arr：要保存的数组。

➤ 读写二进制文件

```
▶ import numpy as np
A = np.arange(16).reshape(2, 8)
print(A)
np.save("C:/2020-2021(2)/bigdata/jupyter code/A.npy", A)
np.load("C:/2020-2021(2)/bigdata/jupyter code/A.npy")
```

```
[[ 0  1  2  3  4  5  6  7]
 [ 8  9 10 11 12 13 14 15]]
```

```
]: array([[ 0,  1,  2,  3,  4,  5,  6,  7],
         [ 8,  9, 10, 11, 12, 13, 14, 15]])
```

➤ 写文本文件

numpy 中的 **savetxt()** 函数用于将数组中的数据存放到文本文件中，**savetxt()** 的语法格式如下：

numpy.savetxt(filename, arr, fmt='%%.18e', delimiter=' ', newline='\n')

filename: 存放数据的文件名。

arr: 要保存的数组。

fmt: 指定数据存入的格式。

delimiter: 数据列之间的分隔符，数据类型为字符串，默认值为' '。

newline: 数据行之间的分隔符。

➤ 读文本文件

`np.loadtxt(fname, dtype=, comments='#', delimiter=None, converters=None, skiprows=0, usecols=None, unpack=False, ndmin=0, encoding='bytes')`

参数	使用说明
fname	str , 读取的 CSV 文件名
delimiter	str , 数据的分割符
usecols	tuple(元组) , 执行加载数据文件中的哪些列
unpack	bool , 是否将加载的数据拆分为多个组, True 表示拆, False 不拆
skiprows	int , 跳过多少行, 一般用于跳过前几行的描述性文字
encoding	bytes , 编码格式



➤ 读写文本文件

```
❏ import numpy as np
b = np.arange(10).reshape(2, 5)
print(b)
np.savetxt("C:/2020-2021(2)/bigdata/jupyter code/b.txt", b)
print(np.loadtxt("C:/2020-2021(2)/bigdata/jupyter code/b.txt"))
#将数组元素保存为浮点数，以逗号分隔
np.savetxt("C:/2020-2021(2)/bigdata/jupyter code/c.txt", b, fmt="%f", delimiter=",")
#load时也要指定以逗号分隔，指定要读取的数据类型为浮点型
np.loadtxt("C:/2020-2021(2)/bigdata/jupyter code/c.txt", dtype="f", delimiter=",")
```

```
[[0 1 2 3 4]
 [5 6 7 8 9]]
[[0.  1.  2.  3.  4.]
 [5.  6.  7.  8.  9.]]
```

```
9]: array([[0., 1., 2., 3., 4.],
          [5., 6., 7., 8., 9.]], dtype=float32)
```